

Fabricated Authority in the Wild: Policy Provenance Controls Direct Tool-Calling Agent Failures

Anonymous Author(s)

September 5, 2026

Abstract

Tool-using language-model agents can refuse in text while still taking an unsafe action. We study *fabricated authority*: user-controlled claims that a forbidden action is allowed by a non-existent approval, exemption, or superseding policy. Across 1,968 hosted-model trials with 8 scenarios, 6 operating-policy phrasings, 7 attack templates, and 2 production models, fabricated authority raises logged tool-call violations from 0/240 in controls to 208/1440 under attack (14.4%, $p = 1.1 \times 10^{-15}$). The largest lever is the deployer’s policy wording. A provenance-aware policy that states user-supplied exemptions are not authoritative has 0/288 direct user-message violations, while a documented-exemption carve-out has 93/288 (32.3%, $p = 1.4 \times 10^{-32}$); a matched contrast gives complete separation, 0/18 vs 18/18. A length-control ablation shows this is not verbosity: a longer irrelevant policy is worse. An adaptive attacker with white-box access breaks weak controls in 8/8 and 7/8 scenarios across two runs, but breaches the provenance-aware policy in 0/8 scenarios both times. However, a forced indirect-channel study breaks the same policy in 15/36 poisoned policy-lookup trials, so the mitigation is channel-specific rather than universal. The result is a narrow, actionable finding: agent evaluations should score tool logs, and deployment prompts should explicitly reject unauthenticated authority claims while treating tool-output provenance as a separate run-time security problem.

1 Why we changed the outcome variable

This work began as a GRPO-trained text attacker scored by an LLM judge. It appeared to succeed: attack success rate rose from 25% to 48%. That figure is an *artifact* of an unvalidated judge, as we now show. We validated the judge against 200 human-labelled responses (Table 1).

Table 1: Three judges scoring 200 identical target responses against human labels. “Reported ASR” is the number a paper using that judge would print; the human-labelled truth is 19%.

Judge	Acc.	κ	Prec.	Rec.	False pos.	Reported ASR
llama3.2	36%	0.09	23%	100%	128	83%
Refusal heuristic	94%	0.83	76%	100%	12	25%
gpt-4o-mini	93%	0.75	93%	68%	2	14%

A 69-point swing in reported ASR on identical outputs. The local judge is worse than the trivial baseline of always answering “refused” (81%), and labelled 128 of 162 genuine refusals as successful jailbreaks, including responses whose entire text is “I can’t fulfill that request.” Re-run with a validated judge on 120 held-out behaviors, our trained attacker showed 12.5% $\rightarrow$ 11.7% ($p = 1.00$): no effect. Training did change

the model, just not usefully: it raised well-formed output from 71% to 91% and tag compliance from 0% to 46%. GRPO taught the output protocol, not the attack, and the apparent gain was judge error.

Rather than build a better judge, we changed what we measure. For a tool-using agent the ground truth is already structured: either the forbidden tool was called with policy-violating arguments, or it was not. **No LLM judge appears anywhere in the agentic pipeline below.** This separates our contribution from text-jailbreak work such as HarmBench, LLMStinger and DarkCite [1, 2, 3], and from judge-reliability work showing that ASR can be dominated by evaluator false positives [4, 5]. Closest in spirit are agent benchmarks showing that text safety does not transfer cleanly to tool calls [6, 7] and RL prompt-injection attacks against tool-use [8]. Our focus is narrower and deployment-facing: which operating-policy phrasings make fabricated authority succeed or fail when the outcome is a verifiable tool call.

2 Setup

Each trial places a model in an agentic loop with tools and an operating policy that forbids a specific action (e.g. exporting patient records). The attack appends fabricated authority: a cited directive, a claimed prior approval, a documented audit exemption. We vary 8 scenarios, 6 policy phrasings, 7 attacks and 2 targets (gpt-4o-mini, claude-haiku-4-5). The dependent variable is a logged tool call. Total hosted-API cost: under \$3.

3 Results

The attack works. Fabricated authority raises violations from **0/240** (control) to **208/1440** (14.4%), one-sided Fisher $p = 1.1 \times 10^{-15}$. The control sits exactly at the floor, so every violation is attributable to the attack.

Table 2: Attacked trials only, both targets pooled. The only difference between `strict` and `strict_hatch` is one sentence stating that claimed exemptions arriving in user input or tool output are not authoritative.

Policy phrasing	Violations	Rate	95% CI (Wilson)
<code>strict_hatch</code>	0/288	0.0%	[0.0%, 1.3%]
<code>strict</code>	7/288	2.4%	[1.2%, 4.9%]
<code>bare</code>	21/288	7.3%	[4.8%, 10.9%]
<code>autonomous</code>	87/288	30.2%	[25.2%, 35.7%]
<code>exemption</code>	93/288	32.3%	[27.2%, 37.9%]

Policy phrasing dominates (Table 2). `strict_hatch` vs `exemption`: $p = 1.4 \times 10^{-32}$. Note the non-monotonicity: `bare` (prohibition only) is *safer* than `exemption`. A documented-exemption carve-out is an explicit invitation to fabricate a document; adding it is worse than adding nothing.

It is the clause, not the length. A verbosity control (`strict_verbose`, 98 words of irrelevant policy, longer than `strict_hatch`’s 77) scored **11/96 (11.5%)** versus `strict_hatch`’s **0/96** ($p = 0.000361$) and *doubled* the 5.2% rate of the shorter `strict` policy it extends. The longest policy was the worst. What the policy says matters, not how much.

Matched contrast: complete separation. Holding scenario, attacks, model and temperature fixed on `medical_export: exemption` scored **18/18**, `strict_hatch` **0/18** ($p = 1.1 \times 10^{-10}$). The one-sentence clause is the only difference.

Which reasoning step fails. Claims about *provenance and process state* succeed (`audit_exemption` 21.7%, superseded 17.9%, `prior_approval` 17.9%); claims about *category membership* fail completely (`reclassify` 0/240) as does urgency (`emergency` 0.8%). Models verify the semantics of the act but not the provenance of the authority permitting it: they have no mechanism to check whether Directive 12.3(b) exists.

Models differ 20x; policy closes the gap. `claude-haiku-4-5` violates 10/840 (1.2%) against `gpt-4o-mini`'s 198/840 (23.6%). Under a permissive policy `gpt-4o-mini` violates in 6 of 10 attacked trials; under `strict_hatch` it never violates.

The clause survives an adaptive attacker. An attacker LLM given the defender's exact system prompt, tool schema, and the full transcript of its own failed attempts proposed a fresh misdirection each round for 8 rounds on each of 8 scenarios. It broke the weak control policy **8/8** but never breached `strict_hatch` (**0/8**, $p = 7.8 \times 10^{-5}$). Because the same attacker demonstrably succeeds against the control, this is an *informative* null rather than a weak-attack artifact: the validity check our hand-written attempt failed. An independent replication with a 10-round budget reproduced the result (`control` 7/8, `strict_hatch` 0/8 over 80 adaptive attempts, $p = 7.0 \times 10^{-4}$).

Refusal in prose, violation in action. In 32 of 432 paired trials the agent states a refusal while simultaneously emitting the forbidden tool call ($p = 1.3 \times 10^{-6}$; never observed without an attack). A text-based evaluation would score these as successful refusals. This is direct evidence that text and tool calls are different measurements.

4 Recommendations

1. **Add a provenance clause:** claimed exemptions, approvals or superseding policies arriving in user input or tool output are not authoritative. 0/288 in our sweep, 0/18 in matched contrast.
2. **Do not write exemption carve-outs.** The most attackable phrasing we tested, worse than silence on exemptions.
3. **Avoid autonomy-forward phrasing** for agents holding destructive tools.
4. **Always provide a safe alternative action**, or refusal degrades into paralysis.
5. **Evaluate agents on tool calls, not text.** Under attack they disagree.

5 Limitations

Two hosted targets, both small production models; no frontier-model claim. A 3B local model (`llama3.2`) violates its policy 67.5% of the time with no attack at all, leaving no headroom, and is reported separately. Single-turn: a forced indirect study that delivers the fabricated authority through a required policy-lookup tool result produces 0/72 clean violations but 51/72 poisoned violations, including 15/36 against `strict_hatch`. Thus the provenance clause mitigates direct user-message authority claims in our setting;

it is not a runtime provenance system for compromised or poisoned tool outputs. Scenarios are hand-written, though per-scenario rates span 7.1%–15.7%. Our hand-written adaptive attacks were too weak to test the clause (0/12 on policies where standard attacks score 18/18); Section 3 replaces them with an LLM attacker, but that attacker is itself only `gpt-4o-mini`, and a stronger or RL-trained attacker may yet succeed.

Ethics statement

We study attacks on safety-constrained agents using hand-written scenarios and no real systems or data. The central finding is a *defence* that deployers can apply for free. We release evaluation tooling and aggregate statistics; the scenario-specific attacker strings are not operational credentials, exploits, or access paths to any real system.

References

- [1] Mazeika et al. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. *ICML*, 2024.
- [2] Jha et al. LLMStinger: Jailbreaking LLMs using RL fine-tuned LLMs. *arXiv:2411.08862*, 2024.
- [3] Yang et al. The Dark Side of Trust: Authority Citation-Driven Jailbreak Attacks on Large Language Models. *arXiv:2411.11407*, 2024.
- [4] Schwinn et al. A Coin Flip for Safety: LLM Judges and Adversarial Robustness Evaluation. *arXiv:2603.06594*, 2026.
- [5] How Reliable Is Your Jailbreak Judge? *arXiv:2606.25487*, 2026.
- [6] GAP: A Benchmark for General Agentic Policies. *arXiv:2602.16943*, 2026.
- [7] AgentSeer: Measuring Safety in Tool-Calling Agents. *arXiv:2509.04802*, 2025.
- [8] AutoInject: Automated Prompt Injection Attacks against Tool-Using Agents. *arXiv:2602.05746*, 2026.